

UNIVERSIDAD NACIONAL DEL SUR

Proyecto 2: Iluminación, Materiales y Texturas

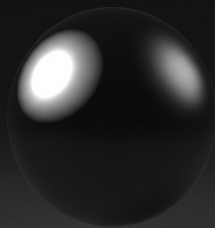
Computación Gráfica

Leonel Pizzuti • Matías Vanzini • Alejo Fernández
Integrantes

Objetivos

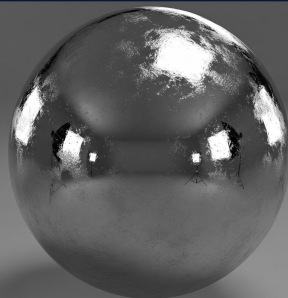
- Aplicación y uso de texturas (mapeo 2D, procedurales y mapeo de normales).
- Familiarización con la creación y manipulación de distintos tipos de luz.
- Simulación de distintos materiales a través de diferentes modelos de luz y uso de texturas.
- Creación de escenas realistas haciendo uso de estos recursos.

Shaders Implementados



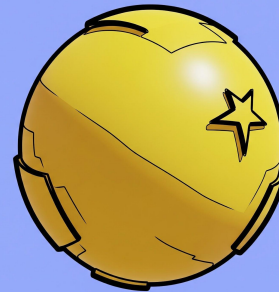
Blinn-Phong

Modelo de reflexión empírico que aproxima la iluminación especular mediante el vector medio, ofreciendo un equilibrio eficiente entre realismo y rendimiento computacional.



Cook-Torrance

Modelo basado en microfacetas para renderizado físicamente preciso (PBR). Simula rugosidad, fresnel y conservación de energía en superficies metálicas y dieléctricas.



Toon Shader

Técnica de renderizado no fotorealista (NPR) que cuantiza los colores para crear un estilo de dibujo animado con bordes definidos.

Shader Toon: Detalles Técnicos

■ Cel Shading

Es una técnica de renderizado no fotorealista que cuantiza la iluminación suave en bandas discretas de color.

Simula el estilo de los dibujos animados tradicionales, reemplazando los gradientes naturales por sombras de bordes duros y colores planos.

■ Inverted Hull

Método utilizado para generar los contornos o "outlines" del objeto.

Consiste en duplicar la malla, expandir sus vértices a lo largo de las normales, invertir las caras y renderizar solo las superficies internas en un color sólido (generalmente negro).

Tipos de Luz Soportados

Los tres shaders implementados ofrecen soporte integral para tres tipos de fuentes lumínicas, configurables desde el inspector.

Luz Direccional

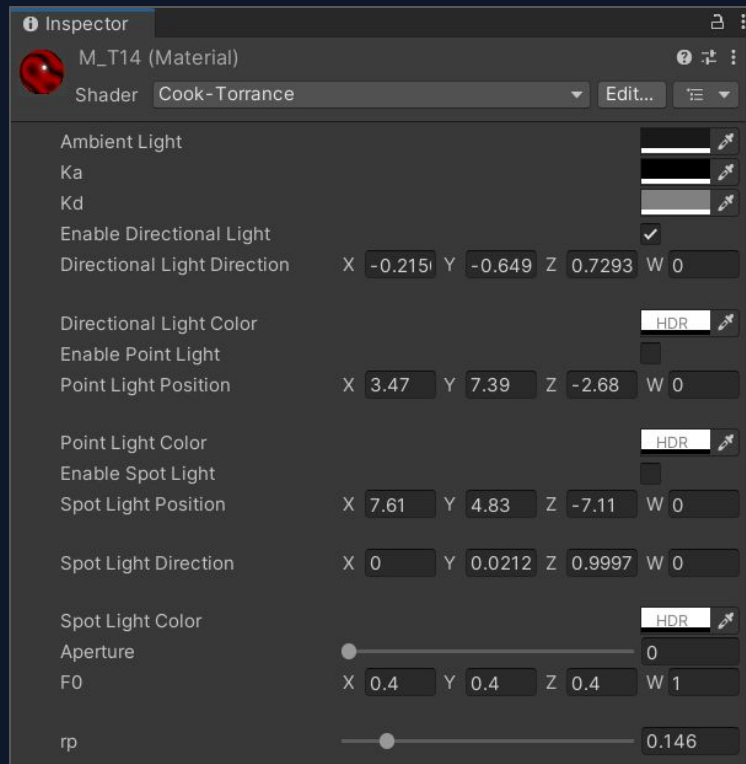
- Simula una fuente de luz infinitamente lejana (como el Sol).

Luz Puntual

- Emite luz en todas las direcciones desde un punto específico.

Luz Spot

- Proyecta un cono de luz limitado por un ángulo y dirección.

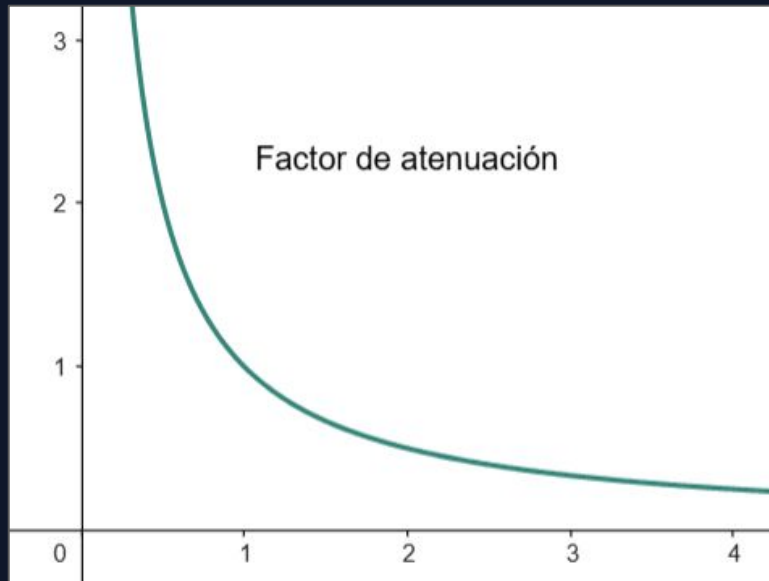


Atenuación de la Luz

A diferencia de la luz direccional, las fuentes locales presentan una pérdida de intensidad natural basada en la cercanía al objetivo.

La **luz puntual** y la **luz spot** calculan su intensidad final multiplicando el color base por un factor de atenuación.

A mayor distancia entre la fuente y el fragmento, menor es la energía lumínica recibida, simulando el comportamiento físico real.



Cálculo del Color Final

El motor de renderizado procesa la iluminación de forma aditiva para determinar la apariencia de cada píxel.

- El color final de un **fragmento** depende de la **contribución individual** de cada fuente de luz presente en la escena.

```
float3 contribAmbient = _AmbientLight.rgb * _Ka.rgb;
float3 contribDir = CalcularDireccional(N, V, _DirLightDirection.xyz, _DirLigh
float3 contribPoint = CalcularPuntual(N, V, i.worldPos, _PointLightPosition.xy
float3 contribSpot = CalcularSpot(N, V, i.worldPos, _SpotLightPosition.xyz, _S

float3 finalColor = contribAmbient + contribDir + contribPoint + contribSpot;

return fixed4(finalColor, _Kd.a);
```

Control de Escena y Luces

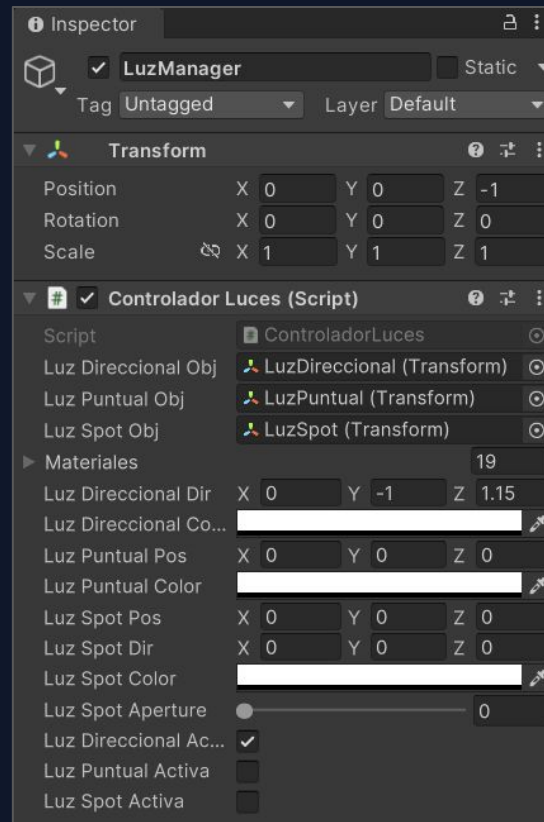
La gestión centralizada permite una interacción dinámica entre las fuentes de luz y los materiales de la escena.

■ Script LuzManager

Control central que sincroniza la reacción de todos los materiales en ambas escenas a la totalidad de las luces simultáneamente.

■ Posicionamiento Dinámico

Capacidad de mover y transformar las luces vinculándolas a objetos físicos posicionados dentro de la escena.



Sistema de Cámaras

■ Cámara Orbital

- **Click derecho + mouse:** Rotación.
- **Scroll:** Control de zoom.
- **Flechas Izq/Der:** Recorrer objetos.
- **Espacio:** Volver al centro.

■ Cámara Primera Persona

- Atada al suelo (movimiento 2D).
- **Controles:** WASD + mouse.

■ ControladorCamara

Alterna cámaras con tecla 'C' y gestiona la lista de objetos de interés.



Texturas Procedurales

Generación algorítmica de superficies detalladas sin el uso de mapas de bits tradicionales.

■ ¿Qué son las texturas procedurales?

Son texturas creadas mediante **funciones matemáticas** en lugar de imágenes estáticas. Esto permite obtener un nivel de detalle infinito sin perder resolución al acercarse.

Ventajas clave:

- **Ahorro de memoria:** Solo requieren código, no archivos pesados.
- **Flexibilidad:** Parámetros editables en tiempo real.
- **Sin costuras:** Se pueden mapear sobre cualquier geometría de forma continua.

Texturas Procedurales: Tablero de Ajedrez

■ ¿Qué es el patrón Checkerboard?

Es una de las texturas procedurales más básicas, generada mediante una función matemática que alterna colores basándose en las coordenadas (u, v) del objeto.

Características principales:

- Basada en funciones de signo o módulo.
- Genera un tablero de ajedrez.



Texturas Procedurales: Fracturas de Voronoi

■ Definición y Uso

Las texturas de Voronoi se generan mediante la partición del espacio en regiones basadas en la distancia a puntos específicos.

En computación gráfica, son fundamentales para simular patrones orgánicos y fracturas naturales como:

- Grietas en pavimentos o piedras.
- Estructuras celulares orgánicas.
- Patrones de escamas o piel seca.



Texturas Procedurales: Domain Warping

■ ¿Qué es el Domain Warping?

El Domain Warping es una técnica donde se distorsiona el dominio de una función de ruido utilizando otra función de ruido como entrada.

Esta técnica permite crear:

- Efectos de fluidos gaseosos o humo.
- Formaciones geológicas complejas.
- Patrones de mármol o madera realista.



Efectos Especiales



Controlador de Luces

Implementamos una **animación simple** para hacer a la escena más dinámica.

El color de la **luz spot** transiciona suavemente a lo largo del tiempo utilizando el controlador.

```
void Update()
{
    if (Input.GetKeyDown(KeyCode.T))
    {
        efectoRGBActivo = !efectoRGBActivo;

        if (!efectoRGBActivo) luzSpotColor = colorSpotOriginal;
    }

    if (efectoRGBActivo)
    {
        float matiz = Mathf.Repeat(Time.time * velocidadRGB, 1f);
        luzSpotColor = Color.HSVToRGB(matiz, 1f, 1f);
    }
}
```

Inconvenientes



Blinn-Phong



Cook-Torrance



ShaderToon

```
SubShader
{
    Tags { "Queue"="Transparent" "RenderType"="Transparent" }
    Blend SrcAlpha OneMinusSrcAlpha
    ...
}
```

Inconvenientes



Inconvenientes



Blinn-Phong



Cook-Torrance



ShaderToon



Blinn-Phong-
Transparente



Cook-Torrance-
Transparente



ShaderToon-
Transparente

SubShader

```
{  
    Tags { "Queue"="Geometry"  
           "RenderType"="Opaque" }  
    ...  
}
```

SubShader

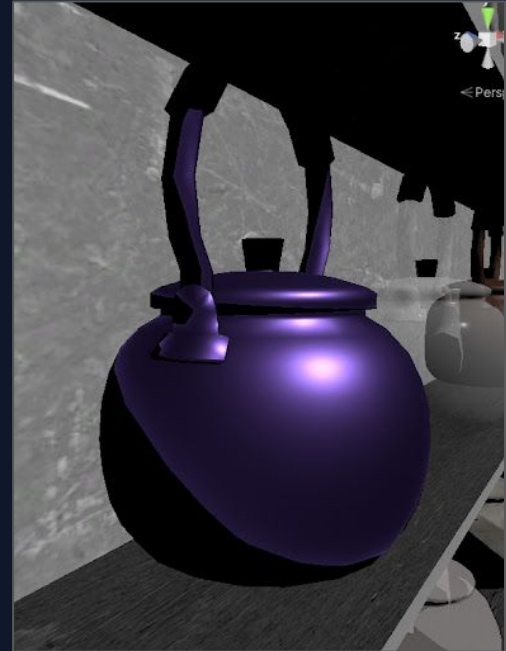
```
{  
    Tags { "Queue"="Transparent"  
           "RenderType"="Transparent"  
    }  
    Blend SrcAlpha OneMinusSrcAlpha  
    ...  
}
```

Inconvenientes



Inconvenientes

```
float3 CalcularDireccional(  
    float3 N,  
    float3 V,  
    float3 lightDir,  
    float3 lightColor,  
    float3 baseColor)  
{  
    ...  
    float NdotL = max(0.0, dot(N, L));  
    float3 diffuse = baseColor * NdotL;  
    float3 specular = CalcularEspecular(V, H, L, N);  
    return lightColor * (diffuse + specular);  
}
```



Inconvenientes

```
float3 CalcularDireccional(  
    float3 N,  
    float3 V,  
    float3 lightDir,  
    float3 lightColor,  
    float3 baseColor)  
{  
    ...  
    float NdotL = dot(N, L);  
    float3 diffuse = baseColor * smoothstep(-0.2,  
        0.2, NdotL);  
    float3 specular = CalcularEspecular(V, H, L, N)  
        * smoothstep(-0.2, 0.2, NdotL);  
    return lightColor * (diffuse + specular);  
}
```



Bibliografía y Recursos



UNITY TOON SHADER

Manual de Instalación v0.6.1
docs.unity3d.com/Packages/com.unity.toonshader



TEORÍA DE ILUMINACIÓN

Cook-Torrance y Blinn-Phong
Material de cátedra - Computación Gráfica



GENERACIÓN PROCEDURAL

Google Gemini AI
Asistente para funciones matemáticas y lógica



ASSETS: TEXTURAS

Poly Haven Library
polyhaven.com/textures



MODELOS 3D

Sweet Home 3D - Objetos
sweethome3d.com/es/modelos-3d-gratuitos

Fin
